

Bachelor Thesis

Nsak as Framework for Scenario Based Network Security

Course of study	Bachelor of Science in Computer Science
Author	Frank Gauss (gausf1) and Lukas von Allmen (vonal3)
Advisor	Wenger Hansjürg

Version 1.0 of March 9, 2026

Abstract

One-paragraph summary of the entire study – typically no more than 250 words in length (and in many cases it is well shorter than that), the Abstract provides an overview of the study.

Contents

Abstract	ii
1. Introduction	1
1.1. Motivation	1
1.2. Project Goals	1
1.2.1. Research Goals	2
1.2.2. Framework	3
1.2.3. Scenarios	5
1.2.4. Evaluation Goals	7
2. Related Research	8
2.1. Comparison Nsak - Caldera	8
3. Concepts - Framework	9
4. Methodic	10
5. Implementation	11
6. Evaluation	12
7. Discussion	13
8. Conclusion	14
Bibliography	15
List of Figures	16
List of Tables	17
Glossary	18
A. Project Management	19
A.1. Agile Management Process	19
A.2. Definition of Ready (DoR)	19
A.3. Definition of Done (DoD)	20
A.4. Project Structure	20

A.5. Planning and Estimation	21
A.5.1. Risk Assessment	21
A.6. Self-Hosted LLM	21
A.6.1. Hardware Setup	22
A.6.2. System Setup	22
A.6.3. Evaluation	23

1. Introduction

Computer networks are constantly exposed to evolving cyber threats ...

The *Network Swiss Army Knife (NSAK)* is a framework designed to execute orchestrated security scenarios consisting of reusable attack drills in a controlled network environment. The framework aims to support automated security testing and adversary emulation on network edge devices.

The development of the NSAK framework originates from previous project work, in which the core architecture and the basic scenario execution model were developed. This thesis builds upon this foundation and extends the framework by investigating how AI agents can support automated red and blue team scenarios.

...

1.1. Motivation

Hypothesis: AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the NSAK framework and assisting in the evaluation of detection results.

1.2. Project Goals

For a goal to be considered **S.M.A.R.T.**, it must fulfill the following criteria:

- ▶ **Specific:** What do we want to achieve?
- ▶ **Measurable:** What are the success criteria?
- ▶ **Achievable:** How do we plan to achieve this goal?
- ▶ **Relevant:** Why is it important to achieve this goal?
- ▶ **Time-bound:** Goals categorized as *must* have to be achieved within the thesis, while *should* or *could* goals are considered optional or nice to have.

1.2.1. Research Goals

Compare Frameworks (must)

To identify the USP (unique selling point) of the NSAK framework, we will analyze and compare existing security emulation frameworks. First, relevant frameworks such as MITRE Caldera and Atomic Red Team will be identified and reviewed. Based on this analysis, the identified concepts will be compared with the architecture of the NSAK framework. The comparison will focus on how scenarios and drills are defined, how configuration and parameter handling are structured, how reusable components are organized, how automation is achieved, and which AI capabilities they provide.

The analysis and comparison will be based on the following properties:

- ▶ Concepts
- ▶ Modularity
- ▶ Configurability
- ▶ Automation
- ▶ AI Capabilities

Success criteria:

- ▶ At least two security emulation frameworks are identified and reviewed
- ▶ A matrix comparing the specified properties of the selected frameworks is created
- ▶ The USP of the NSAK framework is identified

AI Integration (must)

The goal of this research is to investigate how AI (artificial intelligence) can support automated security testing and adversary emulation. The research evaluates potential integration points where AI could enhance the scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework.

Success criteria:

- ▶ How is AI currently used in cybersecurity automation?
- ▶ Which tasks in adversary emulation can be supported or automated by AI?
- ▶ How could AI interact with the NSAK scenarios and drills (e.g., Agents, MCP)?

- ▶ What architectural changes would be required to integrate AI into NSAK?

1.2.2. Framework

Configuration Management (must)

The goal is to extend the NSAK framework, so that scenarios and drills can expose configurable parameters to increase the flexibility of scenarios and reusability of drills. Parameters can be provided via command-line arguments or YAML files and are resolved by the framework into a unified run configuration used for scenario execution.

Success criteria:

- ▶ Scenarios and drills can expose configurable parameters
- ▶ The available parameters can be listed by the CLI with the `--help` option
- ▶ Parameters can be provided via CLI arguments or YAML files
- ▶ The framework resolves the inputs into a run configuration (internal data structure).

REST API (TBD)

Adding a REST API to the NSAK framework was already proposed in the predecessor project, as it greatly enhances interoperability with other tools. The priority of this goal depends heavily on the outcome of the goal 1.2.1 and on whether we want to fulfill the goal 1.2.2, which is prioritized as “could”. If the evaluated AI integration requires a REST API (e.g., MCP), this goal needs to be prioritized as “must”; it will be prioritized as “could”.

Success criteria:

- ▶ A REST API specification with feature parity to the CLI is created.
- ▶ The REST API is implemented according to the specification.
- ▶ The API is documented (Open API) and usable by external clients.
- ▶ The REST API remains optional and does not replace the CLI-based interaction.
- ▶ Authentication and Authorization are explicitly out of scope.

MCP Server (TBD)

The MCP Protocol allows the AI to interact with applications in a standardized way - and could therefore be necessary for implementing AI-based scenarios. If the outcome of the research task 1.2.1 highlights the necessity to specify/expose an MCP-Server to interact seamlessly with the NSAK framework, the implementation is a (must), otherwise a (could).

- ▶ A MCP-Server is specified and implemented
- ▶ Documentation how to connect to the MCP is provided.
- ▶ A form of authentication is provided.

Web GUI (could)

The goal is to design and implement a web-based graphical user interface that allows users to interact with the NSAK framework in a user-friendly way. The Web GUI will enable management of scenarios and drills, triggering scenario execution, and visualization of execution results and system status.

Success criteria:

- ▶ A simple Web GUI that interacts with the REST API has been implemented.
- ▶ The Web GUI displays a list of available scenarios and their states.
- ▶ The scenario's results can be viewed on the Web GUI
- ▶ Scenarios can be configured and executed via the Web GUI
- ▶ Authentication and authorization are explicitly out of scope.

1.2.3. Scenarios

AI – Purple Team (must)

The goal is to implement artificial intelligence according to the result of 1.2.1 and to explore the use of AI to support red, blue, and purple team exercises in the NSAK framework. The AI will be evaluated for its potential to assist in executing attacks in a scenario and analyzing detection results. The findings will help assess how AI could enhance and coordinate red and blue team activities across a network via the NSAK device.

Success criteria:

- ▶ The AI is integrated into NSAK as a scenario or as part of the framework, depending on the result of 1.2.1.
- ▶ The AI can execute CLI or Python tools to directly or indirectly interact with the attached network.
- ▶ An operator can interact with or stop the running AI.
- ▶ The result artifacts are stored and can be presented or reused for further scenarios.

Reproducible Test Environment (must)

The goal is to design and implement a reproducible and safe test environment for the NSAK framework that enables consistent execution and evaluation of scenarios and drills. The environment should provide a stable basis for testing and comparison of manual and AI-assisted scenarios.

Success criteria:

- ▶ The test environment can be set up reproducibly using a defined configuration or setup procedure with Infrastructure as code (e.g., Ansible)
- ▶ NSAK scenarios and drills can be executed consistently within the environment.
- ▶ The environment supports both manual and AI-assisted scenarios.
- ▶ Scenario results can be collected and analyzed for evaluation purposes.

Network Discovery – Red Team (must)

The goal is to implement a reference red team scenario for network discovery within the NSAK framework. The scenario should simulate reconnaissance activities to identify hosts, services, and network characteristics, and the results should be comparable to the AI-Agent approach.

Success criteria:

- ▶ A red team network discovery scenario is implemented in the NSAK framework.
- ▶ Discovered network information is collected as artifacts.
- ▶ The scenario can be executed reproducibly within the test environment.
- ▶ The results can be compared with those generated by the AI-assisted approach.

Intrusion Detection – Blue Team (should)

The goal is to define and implement a blue team reference scenario for intrusion detection within the NSAK framework. The scenario should allow observation and evaluation of suspicious network activity to assess detection capabilities in the test environment and compare the results with those of the AI approach.

Success criteria:

- ▶ A blue-team intrusion-detection scenario is implemented in the NSAK framework.
- ▶ Detection events or alerts can be observed and documented during scenario execution.
- ▶ Detection results can be compared with results from AI-assisted scenario execution.

1.2.4. Evaluation Goals

AI Scenario (must)

The objective is to determine the effectiveness of the AI-based scenario introduced by the goal 1.2.3 within the network environment defined by the goal 1.2.3. The evaluation will assess whether the AI scenario successfully executed the expected red and blue team activities for the provided prompts and the quality of the results.

Success criteria:

- ▶ A methodical approach for the assessment is evaluated and documented.
- ▶ The results of the AI scenario for red and blue team activity are assessed.
- ▶ There is a conclusion on the effectiveness of the AI scenario for red and blue team activities.

AI vs Engineered Scenarios (must/should)

The goal of this evaluation is to compare the quality of AI-based red (network reconnaissance) (must) and blue (network intrusion detection) (should) team scenarios with that of manually engineered scenarios.

The comparison focuses on criteria such as execution reliability, reproducibility, and coverage. The results will help determine the potential benefits and limitations of AI-supported scenarios and provide insights into how AI could enhance future versions of the NSAK framework.

Success criteria:

- ▶ A methodical approach for the comparison is evaluated and documented.
- ▶ The results of the AI scenario are compared against the respective results of the red team scenario (must)
- ▶ The results of the AI scenario are compared against the respective results of the blue team scenario (should)
- ▶ There is a conclusion on the effectiveness of the AI scenario compared to manually engineered scenarios.

2. Related Research

2.1. Comparison Nsak - Caldera

3. Concepts - Framework

4. Methodic

5. Implementation

6. Evaluation

7. Discussion

8. Conclusion

Bibliography

List of Figures

List of Tables

A.1. Hardware configuration of the self-hosted LLM inference server. .	22
--	----

Glossary

This document is incomplete. The external file associated with the glossary ‘main’ (which should be called `thesis.gls`) hasn’t been created.

Check the contents of the file `thesis.gls`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

If you don’t want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- ▶ Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[automake]{glossaries-extra}
```

- ▶ Run the external (Lua) application:

```
makeglossaries-lite.lua "thesis"
```

- ▶ Run the external (Perl) application:

```
makeglossaries "thesis"
```

Then rerun $\text{\LaTeX}$ on this document.

This message will be removed once the problem has been fixed.

A. Project Management

A.1. Agile Management Process

The project followed an interactive development approach using Gitlab for issue tracking and planning. Work items were organized into epics, milestones and issues. Each iteration lasted two weeks.

Issue Lifecycle The workflow used for ticket tracking consisted of the following stages

- ▶ Backlog
- ▶ Todo
- ▶ Q&A
- ▶ Done

A.2. Definition of Ready (DoR)

An ticket is considered ready for implementation when the following criteria are fulfilled.

Clarity

- ▶ Clear and precise title
- ▶ Objective or problem described in two to three sentences
- ▶ Scope clearly defined

Scope

- ▶ Affected module specified (drill, scenario, core, container, frontend)
- ▶ External dependencies identified

Validation

- ▶ Testable acceptance criteria defined
- ▶ Expected behavior clearly specified

A.3. Definition of Done (DoD)

A ticket is considered done when all the following criteria are fulfilled.

Implementation

- ▶ All acceptance criteria fulfilled
- ▶ Container execution verified
- ▶ Pre-commit hooks pass
- ▶ Logging and error handling consistent

Testing

- ▶ Feature or bug fix tested
- ▶ Acceptance criteria verifiable (automated or manual)
- ▶ Relevant edge cases considered
- ▶ Execution reproducible

Documentation

- ▶ README updated if required
- ▶ Architecture changes documented
- ▶ Thesis relevance briefly noted
- ▶ Limitations documented
- ▶ Required documentation stored for thesis or appendix

A.4. Project Structure

Milestones Milestones represent crucial steps toward the overall project goal.

Epics Epics consist of multiple stories and contribute to fulfilling milestones.

Stories Each story is assigned a priority, sprint, milestone and epic. Stories must meet the Definition of Ready before implementation.

A.5. Planning and Estimation

The following estimation strategy was used:

- ▶ 1 hour corresponds to 1 weight point
- ▶ Iteration length: 2 weeks
- ▶ Sprint Planning duration: 1.5 hours per iteration
- ▶ Sprint Review duration: 1 h per iteration
- ▶ Sprint Retro duration: 1 h per iteration

A.5.1. Risk Assessment

Risk Identification

Risk Analysis

Risk Evaluation / Treatment

Risk treatment strategies:

- ▶ Mitigate
- ▶ Transfer
- ▶ Accept
- ▶ Avoid

A.6. Self-Hosted LLM

During the A.5.1 we realized that the costs for AI tokens required to use cloud based LLMs might cause huge costs. As the project team already has existing hardware which is suitable for running smaller LLMs, one of the possibilities to avoid this risk is the usage of self-hosted LLMs. Besides eliminating the costs for AI tokens, at least during the development phase, it also allows the quick evaluation of different AI Models. Depending on the quality and performance

Add
source
for the
cost
claim

of the smaller models running in the self-hosted setup we can consider not using a cloud based LLM at all.

A.6.1. Hardware Setup

Component	Specification
CPU	AMD Ryzen 5 3600 (6 cores / 12 threads, up to 4.2 GHz)
GPU	NVIDIA GeForce RTX 3070 (8 GB VRAM)
System Memory	32 GB RAM

Table A.1.: Hardware configuration of the self-hosted LLM inference server.

The 8 GB VRAM of the RTX 3070 allows us to run models with 7 Billion parameters

A.6.2. System Setup

LLM Model Manager - Ollama

Webserver - Nginx

LLM Chat GUI - Open WebUI

System Hardening

```
1  # Install UFW
2  sudo pacman -Syu ufw
3  sudo ufw status
4
5  # Allow all outgoing connections
6  sudo ufw default allow outgoing
7
8  # Deny all incoming connections per default
9  sudo ufw default deny incoming
10
11 # Allow SSH for remote access
12 sudo ufw allow ssh
13
14 # Allow HTTP for certbot HTTP challenge
15 sudo ufw allow http
16
17 # Allow HTTPS for Open WebUI
18 sudo ufw allow https
19
20 # Allow default Ollama port
21 sudo ufw allow 11434/tcp
22
23 # Make sure to run this command after adding the "allow" rules!
24 sudo ufw enable
25
26 sudo ufw status
```

Listing A.1: UFW configuration

Port Forwarding via Fritzbox DynDNS via Infomaniak

A.6.3. Evaluation